

Reinforcement Learning Project

Anonymous Authors

Abstract—This project explores the application of reinforcement learning, specifically bandit algorithms, to enhance recommendation systems. Bandit algorithms offer a solution to the exploration-exploitation dilemma, balancing uncertainty and maximizing rewards across various domains. Our study implements both multi-arm and contextual bandit algorithms to address the dynamic nature of user preferences and item availability. Preliminary results suggest that contextual bandit algorithms outperform traditional approaches in recommendation accuracy and short-term rewards. We aim to further investigate their impact on user satisfaction and sales metrics. However, the study faces limitations due to dataset availability. In summary, our project aims to leverage contextual bandit algorithms to provide more tailored recommendations while maximizing sales and user satisfaction.

I. INTRODUCTION

Recommendation systems have gained insight in these last years. In this project, we'll look into these systems using Reinforcement learning. More specifically, through bandits algorithms.

The bandit problem is a classic framework used to describe and solve the dilemma of balancing exploration and exploitation in decision-making processes. These problems are encountered in various fields; Advert Placement, Recommendations services, Network routing, clinical trials ...etc. Bandit algorithms are a class of algorithms designed to solve decision-making problems where you have to choose among multiple options (or "arms") with uncertain outcomes. They aim to maximize the total reward over a series of trials, balancing the need to exploit what is already known to be rewarding with the need to explore potentially more rewarding options that haven't been tried much.

The study of these algorithms presents an interesting intersection with RL, it simplifies the complex landscape of RL to a more manageable scope. While RL deals with an agent learning to navigate an environment through sequences of states, actions, and rewards, often with long-term dependencies and effects, the bandit scenario focuses on single-step decision-making with immediate feedback.

One main challenge for recommendations systems are obviously the dynamic nature of both users and items; Users' preferences change over time, and new items are constantly being introduced. Which requires algorithms that are not only good at learning from past interactions but also capable of adapting to new information and changing circumstances rapidly.

To tackle efficiently these challenges, we'll start first by implementing multi-arm bandits and then go on with contextual ones. It's interesting to study them in order to understand how incorporating contextual information can enhance the decision-making process in recommendation systems. By transitioning

from multi-armed bandits to contextual bandits, we can explore how factors such as user preferences, item features, and contextual signals influence the selection of optimal actions, leading to more personalized and effective recommendations. Additionally, studying contextual bandits allows for the exploration of advanced algorithms such as LinUCB, contextual Thompson Sampling and many more. Furthermore, one can push it away and see how these recommendation systems might influence the sales or find out a way to maximise sales using them, also understand their impact on the users (These are our real goals from this project).

Our results showed that indeed the context enhances recommendation accuracy, leading to more tailored and relevant suggestions for users. Furthermore, when compared with traditional multi-armed bandit approaches such as Epsilon-Greedy and UCB, contextual bandit algorithms demonstrated superior performance in terms of short-term rewards. In the coming days, we'll get closer to our real goals and demonstrate that by using contextual bandits, user satisfaction will increase. Moreover, we'll delve deeper into the impact of these recommendation systems on sales and expecting to reveal a significant increase in conversion rates and revenue generation, highlighting the effectiveness of leveraging contextual information for personalized recommendations.

The datasets, in which we want to conduct our study on, are a real limitation in this project, since we couldn't find a proper dataset that would have all the information needed. As a result, we are still exploring various datasets to gather the required data. For now, we have identified two datasets that provide valuable insights into user preferences and item features. However, these datasets may not encompass all the dimensions needed for a comprehensive analysis, so we continue our search for additional datasets to enrich our study and ensure a more robust evaluation of recommendation algorithms.

As mentioned before, two datasets were explored, that's why we'll provide you with two notebooks (the datasets links are provided in the notebooks to download them).

First Notebook.

Second Notebook.

II. BACKGROUND

A. Recommendation Systems:

As seen in the introduction we will work on Recommender systems. As one of the main application of bandits [1], it can be defined as systems that analyze user preferences and behaviors to generate personalized recommendations for products with the aim of maximizing user satisfaction.

B. Multi Arm Bandits:

We take an action $A_t \in \{1, \dots, k\}$ and receive reward $R_t \in \mathbb{R}$: We maximize expected reward (value):

$$q_*(a) = \mathbb{E}_{R_t \sim \nu_a}[R_t \mid A_t = a] \text{ where } R_t \sim \nu_a \Leftrightarrow R_t \sim \nu(R \mid A_t = a)$$

We want to learn about distributions $\nu_1, \dots, \nu_k$. [1]

C. Contextual bandits:

Contextual bandits are a variant of the classic multi-armed bandit problem where each action's reward depends not only on the action taken but also on some context or state. We have a set of products (actions) that we can recommend to a user. Each product may have different features such as price, rating, category, etc. The goal is to recommend the best product to the user at each interaction.

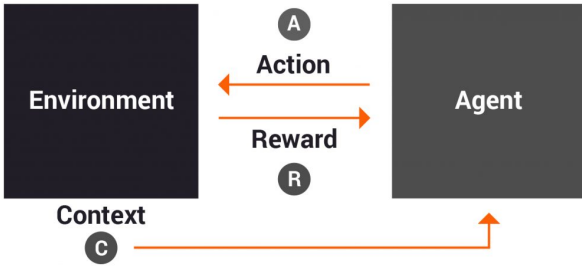


Fig. 1. Contextual Multi Arm Bandits[2]

Here are some of the useful notations:

- **Action Space ($\mathcal{A}$)** is the set of all possible actions or products that can be recommended to the user. Each action corresponds to recommending a specific product.
- **Context Space ($\mathcal{X}$)** who is representing the set of all possible contexts or features that describe the user and the product.
- **Reward Function (R_t)** which is currently the rating, but we will change it to be the CTR (click through rate).
- **Learning Goal:** The goal is to learn a policy ($\pi(x)$) that determines which product to recommend given a context in order to maximize the expected total reward. The policy maps each context to an action, balancing between exploration (trying out different products to gather information) and exploitation (recommending products that are expected to yield high rewards based on past data).

So for this project, we will start by implementing three classic multi-armed bandit algorithms: Epsilon Greedy, Upper Confidence Bound (UCB), and Thompson Sampling. These algorithms will serve as our starting point for building a solid foundation in reinforcement learning for recommendation systems. Here's a brief overview of each algorithm:

- **Epsilon Greedy Algorithm (EP)** is a very well-known strategy to tackle the trade-off between exploitation and exploration. In EP the agent selects the action with the highest estimated value most (You can see this as the arm that has the highest average reward) of the time (exploitation), but occasionally explores other actions with a small probability epsilon (exploration).

- **Upper Confidence Bound (UCB)** is another famous algorithm. It estimates the uncertainty associated with each action's value and chooses the action with the highest upper confidence bound, prioritizing actions with higher potential rewards or less uncertainty.
- **Thompson Sampling**, also known as Bayesian Bandits, is a probabilistic algorithm. It maintains a distribution (for example a beta distribution) over the reward probabilities of each action and selects actions based on sampled probabilities.

Once we have implemented these basic bandit algorithms, we will extend them to incorporate contextual information. Contextual bandit algorithms adapt the action selection process based on contextual features associated with each action. By integrating contextual information into Epsilon Greedy, UCB, and Thompson Sampling, we aim to enhance their recommendation capabilities and provide more personalized and effective suggestions to users.

III. METHODOLOGY/APPROACH

Our methodology revolves around building a recommender system utilizing contextual bandits, with a specific focus on investigating the effectiveness of dynamic rewarding in improving market sales using the Amazon Sales Dataset.

A. Dataset

We will leverage the Amazon Sales Dataset, comprising ratings and reviews for over 1,000 Amazon products. Notably, this dataset lacks user features, prompting us to adopt methods like matrix decomposition, such as Singular Value Decomposition (SVD), to derive user features. Alternatively, more advanced techniques like neural collaborative filtering may be explored if time permits.

Additionally, we have introduced a new table named "explorer," containing user IDs and an "exp score." This score quantifies a user's inclination to explore new products, ranging from 0 to 10, with 10 indicating a high propensity for exploration. We intend to derive this information from the Sales dataset by analyzing user purchase histories and product similarities, albeit recognizing potential limitations in directly correlating purchase behavior with the explorer score.

B. Workflow

The outlined workflow is structured as follows:

- 1) **Bandit Selection:** Determine the number of products to display to the user. For our purposes, we will initially set this to 5.
- 2) **Policy Training:** Train a contextual learning policy using both user and ad features. We plan to evaluate several policies, including LinGreedy, LinTS, and LinUCB.
- 3) **Product Recommendation:** Upon receiving a user ID, the policy will select and recommend 5 products based on the available arms.
- 4) **Simulation of User Interaction:** Simulate user interactions, where each user selects a product to purchase

based on their explorer score. We assume that selecting an arm corresponds to purchasing the associated product.

- 5) **Dynamic Reward Calculation:** Calculate the reward for the policy based on the user's explorer score and the chosen product's price. This dynamic reward mechanism aims to differentiate rewards based on user exploration behavior and the type of product selected.
- 6) **Rating and Feedback Incorporation:** Users will provide ratings for purchased items, with the rating distribution potentially determined by a yet-to-be-decided function. These ratings will subsequently impact the user's explorer score, with personalized item purchases maintaining or potentially decreasing the score, while exploration-based purchases increase it. The precise formula for this adjustment is yet to be determined.

The concept of dynamic rewarding aims to tailor rewards based on user exploration behavior and the nature of the purchased product. We intend to explore how this dynamic approach influences long-term changes in Amazon sales trends.

C. Simulation

Given the absence of real-time future sales and user purchase data, we will construct an artificial environment to simulate user interactions. The simulation process will unfold as follows:

For each time stamp:

- 1) **User Selection:** Randomly select a user ID from the dataset.
- 2) **Product Recommendation:** The policy recommends a set of 5 products to the user.
- 3) **User Interaction:** The user selects one product from the recommendations, representing a purchase in our simplified scenario.
- 4) **Reward Calculation:** Based on the user's explorer score and the selected product's price, the bandit calculates the reward.
- 5) **Rating Assignment:** The user provides a rating for the purchased item, reflecting its perceived quality based on Amazon's product rating.
- 6) **Explorer Score Adjustment:** The user's explorer score adjusts based on the assigned rating and whether the chosen item was personalized or random in nature.

This simulation framework enables us to explore the dynamics of user interactions within the contextual bandit framework, facilitating the evaluation of our proposed dynamic rewarding mechanism.

IV. RESULTS AND DISCUSSION ($\approx$ 1-2 PAGES)

In this section, we present the results of our experiments and provide a detailed discussion on their implications. We analyze the performance of our contextual bandit-based recommender system, focusing on dynamic rewarding, and discuss the factors influencing its effectiveness.

Due to time constraints, we were able to show two useful plots. In figure 2, we show the importance of taking into account context while making decisions. We can see that

the cumulative average reward over time is the highest for LinUCB which takes into account the context while the other 3 policies do not take into account the context. One should mention that the context was generated randomly in the case of LinUCB, but still we have seen an improvement of our reward. Further studies will consider a real context including user features, item features and other contextual information that might be interesting to explore such as the time of the purchase, the location and so as.

In Figure 3, we present the arms chosen by the LinGreedy policy after simulating 200 rounds. This preliminary visualization serves as a verification of our main workflow. However, for a more comprehensive analysis, we recognize the need to increase the number of iterations in our simulation. Currently, with only 200 iterations, gaining meaningful insights is challenging, especially given the large number of products (approximately 1200) in our dataset. Yet, we acknowledge that scaling up the iteration count significantly may result in longer execution times. We plan to delve deeper into this aspect in subsequent analyses, balancing the trade-off between computational efficiency and the depth of insights gained.

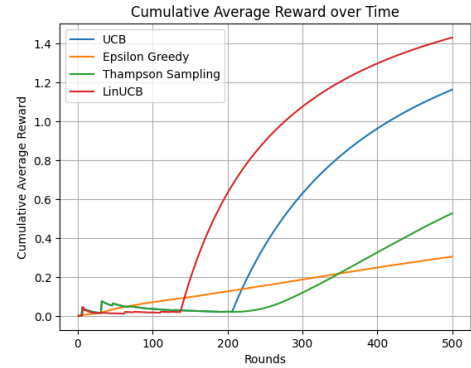


Fig. 2. The impact of having synthetic context on Cumulative Average Reward

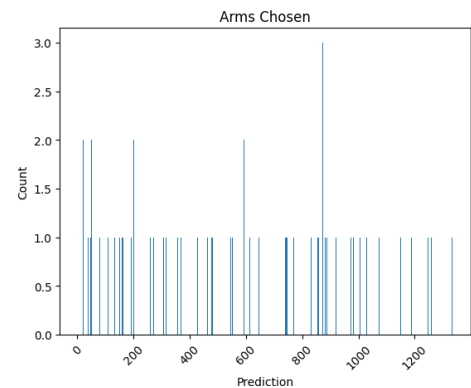


Fig. 3. The arms chosen by LinGreedy after 200 iterations

V. CONCLUSIONS

In conclusion, our project delves into the realm of reinforcement learning algorithms for recommendation systems. We began our exploration by implementing classic multi-armed

bandit algorithms, including Epsilon Greedy, Upper Confidence Bound (UCB), and Thompson Sampling. These algorithms provided a foundational understanding of exploration-exploitation trade-offs in recommendation scenarios.

Building upon this foundation, we extended our study to contextual bandit algorithms, which incorporate contextual information to personalize recommendations further. As mentioned before, due to the dataset limitation, we were not able to gain insights on the user satisfaction and also the revenue profit. We still exploring datasets in the hope to find a proper one in order to implement our objectives.

REFERENCES

- [1] Read. Lecture III - Bandits I: An Introduction. In *INF581 Advanced Machine Learning and Autonomous Agents*, 2024.
- [2] Dattaraj Rao. <https://www.persistent.com/blogs/how-solving-the-multi-armed-bandit-problem-can-move-machine-learning-forward/> posted on June 10, 2020 in Data and Analytics

APPENDIX

A. Alternating Least Squares:

1) *Matrix Factorization*: Matrix factorization decomposes the sparse user-item interaction matrix A into two lower-dimensional matrices: user latent features U and item (product) latent features V . We aim to approximate $A \approx UV^T$, where each entry A_{ij} is approximated as the dot product of the i -th row of U and the j -th row of V^T .

2) *ALS Algorithm*: ALS iteratively optimizes U and V to minimize the reconstruction error between A and UV^T , while regularizing to prevent overfitting.

Algorithm 1: Alternating Least Squares Algorithm

Initialization: Initialize U and V ;

Alternating Optimization:

Fixing U : Update V by solving a least squares problem with U fixed.;

Fixing V : Update U by solving a least squares problem with V fixed.;

Repeat: Alternate between updating U and V until convergence.

3) Advantages of ALS:

- **Scalability**: ALS is scalable and efficient for large-scale datasets.
- **Interpretability**: Learned latent features offer insights into user preferences and item characteristics.
- **Regularization**: ALS incorporates regularization to prevent overfitting.

Applying ALS to the user-item interaction matrix yields dense representations of users and items, facilitating efficient recommendation generation based on latent feature similarity.